CAMPAIGN INTEGRITY — LLD BRIEF

Low-Level Design & Critical Feature Architecture Specification

Track: Martech · Team: MAL 4
Deliverable: One-Page LLD Brief

THE CRITICAL FEATURE INVARIANT *"The campaign that gets scheduled is the exact campaign that was approved."*

```
active_schedule(c) ⇒ current_rev(c).state = APPROVED ∧ decision.rev_id = current_rev(c).id ∧ decision.content_hash = current_rev(c).content_hash
```

1 RELATIONAL SCHEMA & INDICES

Table	Keys & Constraints	Invariant Target
`campaigns`	`id (PK), current_rev_id (FK)`	Lock root: `SELECT FOR UPDATE`
`revisions`	`id (PK), campaign_id, content_hash`	Immutable copy-on-write version tree
`decisions`	`id (PK), revision_id, hash`	Cryptographically bound reviewer approval
`schedules`	`id (PK), rev_id, dec_id`	Partial unique index: 1 active schedule/cmp
`audit_events`	`id (PK), actor_id, rev_id, correlation_id`	Insert-only; UPDATE/DELETE revoked

2 REVISION STATE MACHINE & MUTATIONS

Transition	Trigger Action	Enforced Guard
`DRAFT → IN_REVIEW`	Submit for review	Author only; complete fields
`IN_REVIEW → APPROVED`	Reviewer approves	`reviewer ≠ author ∧ hash_match`
`IN_REVIEW → CHANGES_REQ`	Request changes	Mandatory written rationale note
`APPROVED → SCHEDULED`	Schedule campaign	Single atomic gate checks pass
`APPROVED → SUPERSEDED`	Post-approval edit	Copy-on-write fork; cancels schedule!

3 CLEAN ARCHITECTURE LAYER BOUNDARIES

Mobile Client: Presentation → Use Cases → Domain Helpers → Repositories → SQLite Outbox.
API Monolith: HTTP Transport → Application Services (Tx Boundary) → Domain Core → Persistence Repositories → PostgreSQL.
Dependency Rule: Domain core imports NOTHING from Express, SQL, or React Native.

4 THE ATOMIC SCHEDULING GATE (TX)

```
// ScheduleService.execute() inside PostgreSQL transaction: const c = await tx.query("SELECT * FROM campaigns WHERE id=$1 FOR UPDATE", [id]); if (req.revId !== c.current_revision_id) throw new ConflictError("REVISION_STALE"); const rev = await getRevision(tx, req.revId); if (rev.state !== "APPROVED") throw new ConflictError("APPROVAL_INVALID"); const dec = await getActiveDecision(tx, rev.id); if (dec.content_hash !== rev.content_hash) throw new ConflictError("HASH_MISMATCH"); if (await hasActiveSchedule(tx, c.id)) throw new ConflictError("ALREADY_SCHEDULED"); await insertSchedule(tx, ...); await insertAudit(tx, "CAMPAIGN_SCHEDULED", ...);
```

5 API ENDPOINTS & 409 CONFLICT ENVELOPES

HTTP Command	Key Payload / Header	Deterministic Conflict Response
`POST /campaigns`	`Idempotency-Key`, title, budget	400 Validation Error
`POST /submit`	`Idempotency-Key`	409 REVISION_NOT_DRAFT
`POST /decisions`	`APPROVED | CHANGES_REQ`	403 SELF_REVIEW_PROHIBITED
`PATCH /campaigns/:id`	New budget, audience, objective	Copy-on-write fork (auto-cancel schedule)
`POST /schedule`	`revisionId, expectedHash`	409 REVISION_STALE | HASH_MISMATCH

6 RESILIENCE & DEFENSIBILITY TACTICS

SQLite Command Outbox: Persists user intent locally prior to dispatch; sync engine recovers seamlessly after app kill.
Exactly-Once Idempotency: Client UUID keys guarantee retried requests return cached responses with zero duplicate state mutations.
Derived Not Stored: Approval is computed dynamically. Eliminates mutable boolean drift entirely.